

11장 레지스터 할당

최광훈
전남대학교

개요

- 레지스터 할당은 무한한 임시 변수들을 제한된 실제 레지스터에 배정하는 과정
- 동시에 라이브한 임시 변수들은 같은 레지스터를 사용할 수 없음
- 이러한 제약을 간섭 그래프로 표현
- 간섭 그래프의 노드는 임시 변수이고, 간선은 같은 레지스터를 쓸 수 없는 조건을 의미함
- 그래프 컬러링에서 색은 실제 레지스터에 해당함
- **K**개의 레지스터로 그래프를 색칠할 수 있으면 레지스터 할당 성공
- 불가능하면 일부 값을 메모리에 저장해야 하며, 이를 스�필링(spilling)이라고 함

11.1 Coloring by Simplification

레지스터 할당은 본질적으로 어려운 문제임

- 일반적으로 **NP-complete** 문제임
- 그래프 컬러링 문제도 **NP-complete** 문제임
- 실제 컴파일러에서는 좋은 결과를 내는 선형 시간 근사 알고리즘을 사용

=> Coloring by Simplification 알고리즘

11.1 Coloring by Simplification

전체 알고리즘 구조

- Build -> Simplify -> Spill -> Select -> Start over

단계	역할
(1) Build	간접 그래프 생성
(2)-1 Simplify	낮은 차수의 노드를 제거하며 스택에 저장
(2)-2 Spill	레지스터에 배정하기 어려운 노드를 메모리 후보로 선택(Potential Spill)
(3) Select	스택에서 노드를 꺼내며 색 배정
(4) Start over	실제 spill 이 발생하면 프로그램을 다시 작성하고 이 과정을 다시 반복

11.1 Coloring by Simplification

Build 단계: 간섭 그래프 만들기

- 데이터 흐름 분석(라이브니스 분석)을 통해 각 프로그램 지점에서 동시에 라이브한 임시 변수들의 집합을 구함
- 같은 시점에 동시에 라이브한 임시 변수 쌍마다 간섭 간선을 추가
- (10장 참고)

11.1 Coloring by Simplification

Simply 단계: 간섭 그래프에서 노드 제거하며 스택에 그 노드를 추가

- 목표 기계에 레지스터가 K 개 있다고 가정
- 간섭 그래프에서 어떤 노드 m 에 대하여, 이 노드 m 의 이웃 수가 K 보다 작다면, 즉 $\text{degree} < K$ 이면 그 노드는 일단 제거 가능 (그래프에서 제거하고 스택에 push)
- 왜냐하면 m 의 이웃들이 최대 $K-1$ 개의 색만 사용할 수 있으므로, 남은 색으로 m 을 칠할 수 있기 때문
- 이 과정을 반복
- 어떤 노드를 제거하면 그 이웃들의 degree 가 낮아지고, 그 결과 새롭게 제거 가능한 노드가 생길 수 있음

11.1 Coloring by Simplification

Spill 단계: 간섭 그래프에서 노드 제거하며 스택에 그 노드를 추가 (1/2)

- Simplify 과정 중에 더 이상 $\text{degree} < K$ 인 노드가 없는 경우, 남은 모든 노드의 degree가 K 이상 $\Rightarrow$ significant degree를 가진 노드
 - 이 경우 단순화(Simplify) 휴리스틱이 실패하므로, 어떤 노드를 spill 후보로 선택 (spill 후보 노드를 선택 $\rightarrow$ 그래프에서 제거 $\rightarrow$ 스택에 push $\rightarrow$ Simplify 계속 진행)
 - 선택된 spill 후보 노드 임시 변수는 나중에 레지스터에 배정되지 못하면 메모리에 저장
- (spill 후보 노드를 메모리 저장할지 여부를 실제 색칠할 때까지 결정을 미룸)

11.1 Coloring by Simplification

Spill 단계: 간섭 그래프에서 노드 제거하며 스택에 그 노드를 추가 (2/2)

- **Optimistic Coloring**: **Spill** 후보로 선택되었다고 반드시 실제 **spill**이 되는 것은 아님
- 다음 **Select** 단계에서 다시 색을 배정할 때 그 노드의 이웃들이 우연히 같은 색을 공유하고 있을 수 있기 때문에
- 그러면 사용된 색의 수가 **K**보다 작아져서 해당 노드에도 색(레지스터)을 배정 가능
- 즉, **spill** 후보였지만 색을 배정할 수 있어서 실제 **spill**되지 않음
- 이 방식을 **optimistic coloring**이라고 함

11.1 Coloring by Simplification

Select 단계: 스택에 저장된 노드를 하나씩 꺼내면서 실제 색을 배정

- **Simplify** 단계에서 제거했던 순서의 반대로 노드를 다시 그래프에 추가
- 스택에서 **pop** -> 그래프에 노드 복원 -> 이웃 노드들과 다른 색 선택
- **Simplify** 단계에서 $\text{degree} < K$ 였던 노드는 색을 배정할 수 있음이 보장됨
- **Spill** 후보로 스택에 들어간 노드는 색칠 가능성이 보장되지 않음
- 따라서 **Select** 단계에서 스택에서 **pop**했을 때 이 노드의 이웃들이 이미 K 개의 서로 다른 색을 모두 사용하고 있다면, 더 이상 남는 색이 없어 실제 **spill** 발생
- 해당 임시 변수는 레지스터가 아니라 메모리에 저장해야 함

11.1 Coloring by Simplification

Start Over 단계: 실제 **spill**이 발생하면 프로그램을 다시 작성하고 Coloring by Simplification 과정을 반복

- 프로그램 재작성

- : **spill**된 임시 변수를 사용할 때마다 메모리에서 읽어오고,

- : 정의할 때마다 메모리에 저장하도록 코드를 수정

- 이후 **spill**된 임시 변수가 여러 개의 짧은 **live range**를 가진 새로운 임시 변수들로 나뉨

- 프로그램 재작성 -> Build -> Simplify -> Spill -> Select

- 보통 한두 번 반복하면 더이상 **spill**이 발생하지 않고 **coloring**이 완료됨

11.1 Coloring by Simplification

Spill 임시 변수를
메모리에 저장하도록
프로그램을
재작성하는 예시

// 원래 코드: v의 수명 주기(Live Range)가 끝까지 길게 이어짐
v = a + b; // (1) v의 정의 (Definition)

x = c * 2; // v를 쓰지 않는 중간 코드들
y = d - 1; // 이 구간 동안 v는 레지스터를 차지하며 살아있음

작업 = v / 3; // (2) v의 첫 번째 사용 (Use)
결과 = v + x; // (3) v의 두 번째 사용 (Use)

- spill 대상 v 선택
- 현재 함수 stack frame에 spill slot 할당
- mem_v = frame pointer 기준 offset
- v 정의 직후 store mem_v
- v 사용 직전 load mem_v

// 재작성 후 코드: 메모리 주소 [mem_v]를 활용하도록 변경

// (1) v의 정의 시점 처리
v1 = a + b; // 아주 잠깐 살아있는 새 변수 v1 생성
store v1, [mem_v]; // 정의 직후 메모리에 즉시 백업 (Store)

x = c * 2; // 이제 이 구간 동안은 v에 대한 레지스터가
y = d - 1; // 전혀 필요 없음 (레지스터 절약)

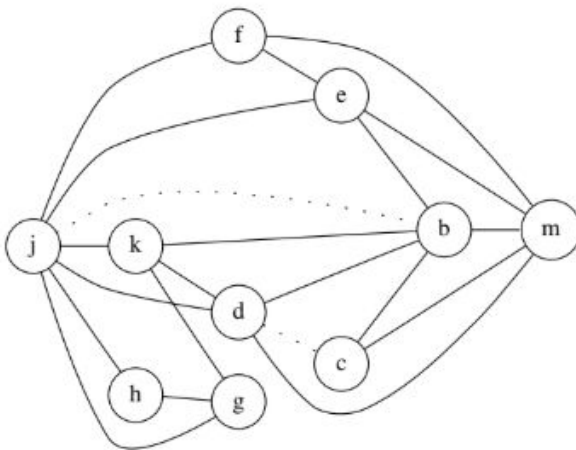
// (2) v의 첫 번째 사용 시점 처리
fetch [mem_v], v2; // 사용 직전 메모리에서 값을 읽어옴 (Fetch)
작업 = v2 / 3; // 아주 잠깐 살아있는 새 변수 v2 사용 후 소멸

// (3) v의 두 번째 사용 시점 처리
fetch [mem_v], v3; // 사용 직전 메모리에서 값을 다시 읽어옴 (Fetch)
결과 = v3 + x; // 아주 잠깐 살아있는 새 변수 v3 사용 후 소멸

11.1 Coloring by Simplification

예제: Graph 11.1 (간섭 그래프 예)

```
live-in: k j
g := mem[j+12]
h := k - 1
f := g * h
e := mem[j+8]
m := mem[j+16]
b := mem[f]
c := e + 8
d := c
k := m + 4
j := b
live-out: d k j
```



d, k, j는 동시에 라이브 (서로 모두 연결)

g, h, c, f의 degree < K (K=4)

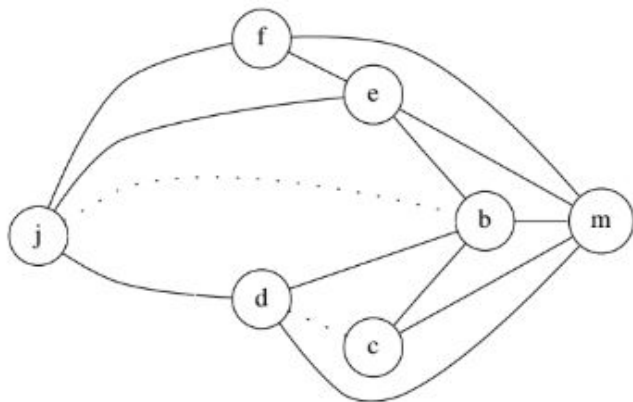
GRAPH 11.1.

Interference graph for a program. Dotted lines are not interference edges but indicate move instructions.

11.1 Coloring by Simplification

예제: Graph 11.2

- g와 h ($K < 4$ 인 노드들 중)를 제거하면, k의 **degree** < K가 되고, k를 그 다음에 제거



GRAPH 11.2. After removal of h, g, k.

11.1 Coloring by Simplification

예제: Graph 11.3

m	1
c	3
b	2
f	2
e	4
j	3
d	4
k	1
h	2
g	4

(a) stack (b) assignment

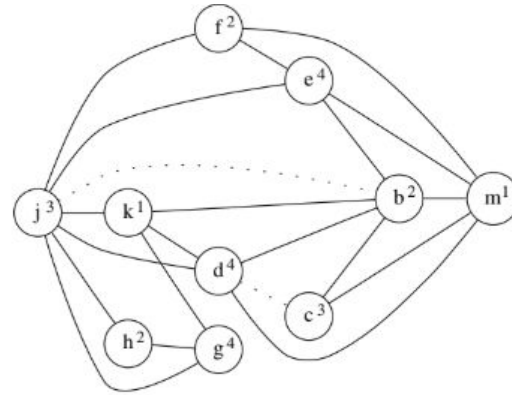


FIGURE 11.3. Simplification stack, and a possible coloring.

11.2 Coalescing

불필요한 **move** 명령어를 제거하기

- **move a, b**에서 **a**와 **b**가 간섭하지 않는다면 두 임시 변수를 하나의 레지스터에 배정 가능하고, 따라서 이 **move** 명령어를 제거할 수 있음
- 두 노드를 하나로 합치는 것을 **coalescing**이라 함
- 노드 **ab**의 이웃 노드는 **a**의 이웃 노드와 **b**의 이웃 노드를 모두 합한 노드들
- 장점: **move** 제거로 불필요한 복사 감소 => 코드 품질 향상
- 단점: 이웃 수가 증가하여 그래프가 더 복잡해져 **K-coloring**이 불가능해질 가능성
=> **move** 명령어 제거 이후 **spill**이 더 많이 발생하여 메모리 접근이 늘어남

11.2 Coalescing

보수적 Coalescing 전략

- Coalescing해도 안전하다고 판단되는 경우에만 노드를 합침
 - => 새로운 **spill**이 발생하는 가능성 감소
 - => 보수적이므로, 실제로는 안전한 Coalescing 기회를 놓칠 수 있음
- Briggs 전략
- George 전략

11.2 Coalescing

Briggs 전략

- ab 의 $\text{degree} \geq K$ 인 이웃을 K 개 미만으로 갖는 경우만 노드 **a**와 **b**를 합침
- **a**와 **b**를 합친 다음에도 ab 의 **significant-degree** 이웃이 충분히 적다면, 나중에 **Simplify** 단계에서 ab 를 색칠할 수 있는 가능성이 높음
- 즉, **coalescing** 후에도 그래프가 색칠 가능할 가능성이 높을 때만 합침

11.2 Coalescing

George 전략

- **a**의 모든 이웃 **t**에 대하여, 다음 조건 중 하나가 성립하면 **a**와 **b**를 합침
- 조건1) **t**가 이미 **b**와 간섭함
- 조건2) **t**의 $\text{degree} < K$

조건1) => **a**와 **b**를 합쳐도 **t**와의 관계가 새롭지 않음

조건2) => **t**의 degree 가 낮다면 나중에 **Simplify** 과정에서 **spill**하지 않고 쉽게 제거 가능

11.2 Coalescing

Freeze 단계

- 보수적 **Coalescing**이 더 이상 진행되지 않을 때도, 아직 **move**가 남아 있을 수 있음
- 이때는 더 이상 **coalescing**을 시도하지 않고 포기함
- 이 **move**를 **freeze**함

레지스터 할당 알고리즘이 계속 진행되도록 하기 위함

11.2 Coalescing

Iterated register coalescing 절차

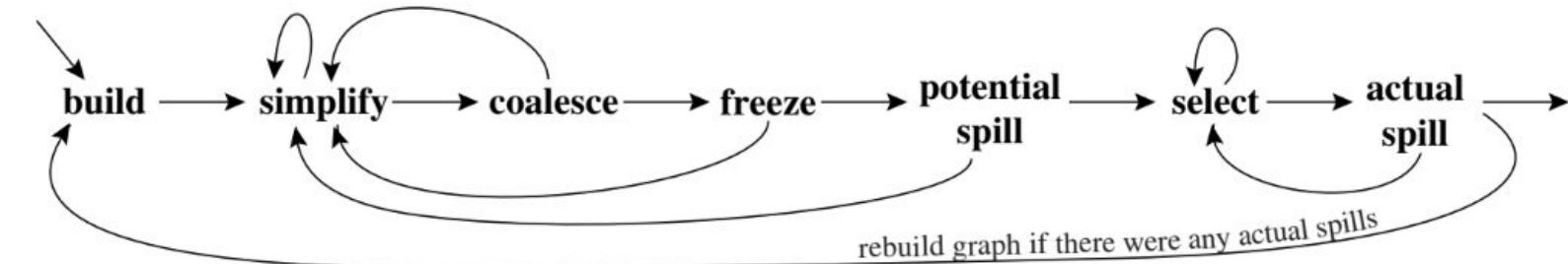
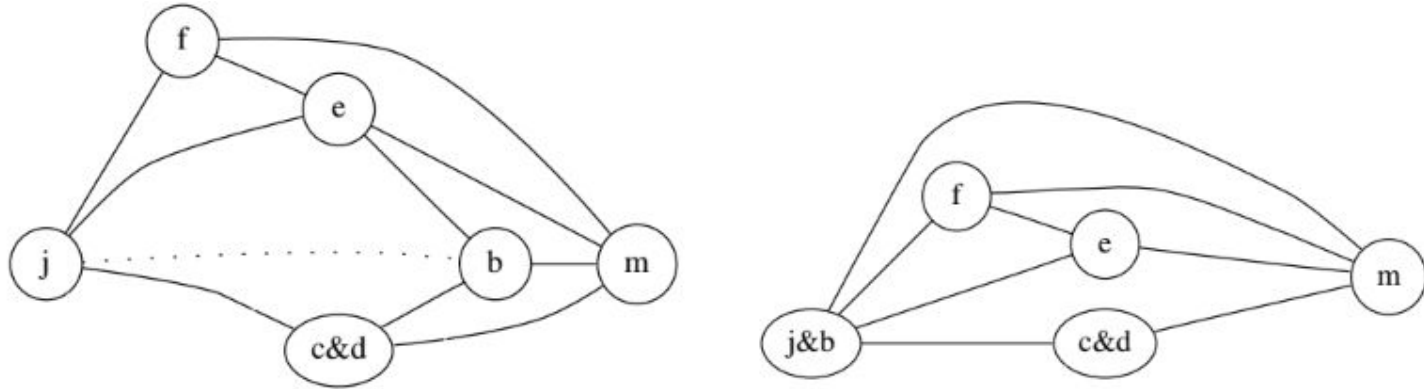


FIGURE 11.4. Graph coloring with coalescing.

11.2 Coalescing

Graph coloring with coalescing 예



GRAPH 11.5. (a) after coalescing c and d; (b) after coalescing b and j.

11.2 Coalescing

Graph coloring with coalescing 예

e	1
m	2
f	3
j&b	4
c&d	1
k	2
h	2
g	1
stack	coloring

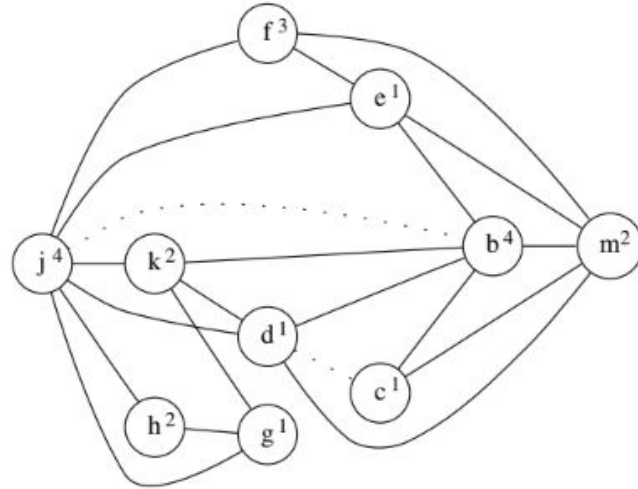


FIGURE 11.6. A coloring, with coalescing, for Graph 11.1.

11.3 Precolored Nodes

일부 temporary는 precolored 상태로 존재

- 일반 임시 변수가 아니라 이미 특정 기계 레지스터를 대표하는 노드
- 예시) 프레임 포인터, 인자 레지스터, 리턴 값 레지스터

Precolored 노드가 필요한 이유?

- 함수 호출 규약을 지키려고 (첫 번째 인자 -> r1, 반환 값 -> rv, 프레임 포인터 -> fp)

11.3 Precolored Nodes

Precolored 노드의 특징

- `Mips.Frame` 모듈에서 해당 레지스터로 매핑되는 임시 변수를 사용
- 이 임시 변수는 나중에 다른 임의의 레지스터로 바꿀 수 없음
- 각 실제 기계 레지스터마다 하나의 **precolored** 노드만 존재함
- 각 **Precolored** 노드는 서로 간섭한다고 가정 (다른 레지스터는 다른 색을 할당)
- **Simplify**하거나 **Spill**할 수 없음 (이미 색이 할당되어 있음)

11.3 Precolored Nodes

레지스터를 임시 변수로 복사하기

- 예) **callee-save** 레지스터 r_7 을 t_{231} 로 옮기면, r_7 을 다른 목적으로 사용 가능



FIGURE 11.7. Moving a callee-save register to a fresh temporary.

11.3 Precolored Nodes

Caller-save vs Callee-save 레지스터

- 임시 변수가 프로시저 호출을 거쳐서 라이브하지 않다면, **caller-save register**에 배정하는 것이 좋고,
- 반면에 여러 프로시저 호출을 거쳐서 계속 라이브하다면, **callee-save register**에 두는 것이 좋은 전략임

이 전략을 레지스터 할당 알고리즘에서 어떻게 반영할까?

- **Coalescing**과 **spilling**으로 자연스럽게 **caller-save/callee-save** 선택이 이루어짐

11.3 Precolored Nodes

Callee-save 레지스터 모델링

- 모든 **callee-save** 레지스터는 프로시저 진입 시점에 라이브한 것으로 간주
- 리턴 명령어에서 사용되는 것으로 간주

=> **callee-save** 레지스터 값을 함수 종료 시까지 보존해야 함을 라이브니스 분석에 반영

11.3 Precolored Nodes

Caller-save 레지스터 모델링

- CALL 명령어는 모든 **caller-save** 레지스터를 **define**하는 것으로 표시
=> CALL 이후 레지스터 값이 덮어써질 수 있다는 의도

11.3 Precolored Nodes

Precolored 노드가 있는 예제

- Precolored node가 있는 경우, 일반 temporary만 있는 경우보다 레지스터 할당 과정이 더 복잡해짐
- Precolored node: 이미 특정 machine register에 고정된 노드
- Callee-save register: 함수가 사용하면 entry/exit에서 보존해야 하는 레지스터
- Spilling: 레지스터가 부족할 때 임시 변수를 메모리에 저장하는 과정

11.3 Precolored Nodes

caller-save 레지스터 : r1, r2 / callee-save 레지스터 : r3

```
int f(int a, int b) {  
    int d=0;  
    int e=a;  
    do {d = d+b;  
        e = e-1;  
    } while (e>0);  
    return d;  
}
```

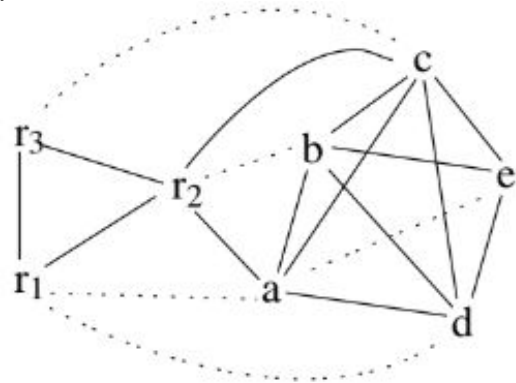
(a)

```
enter:  c ← r3  
        a ← r1  
        b ← r2  
        d ← 0  
        e ← a  
loop:  d ← d + b  
        e ← e - 1  
        if e > 0 goto loop  
        r1 ← d  
        r3 ← c  
return
```

(b)

명령어 선택 결과

c ← r3
...
r3 ← c



(r1, r3 live out)

11.3 Precolored Nodes

$K = 3$

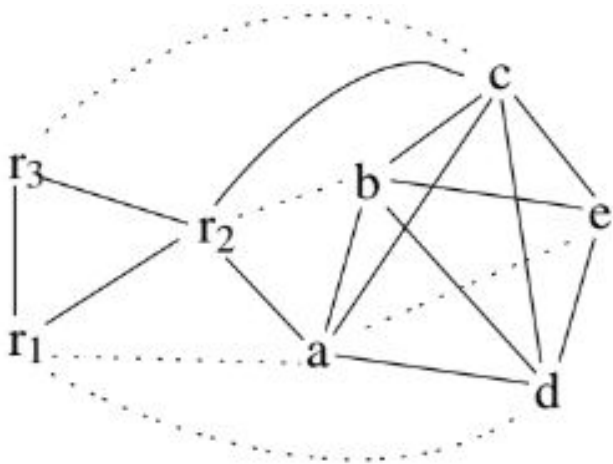
1. 모든 일반 노드 (Precolored가 아닌) $\text{degree} \geq K$

=> Simplify나 freeze 적용할 수 없음

합쳐진 노드는 K 개 이상의 significant-degree 이웃을 가지게 됨

=> Coalesce 적용할 수 없음

따라서 Spill 해야 함

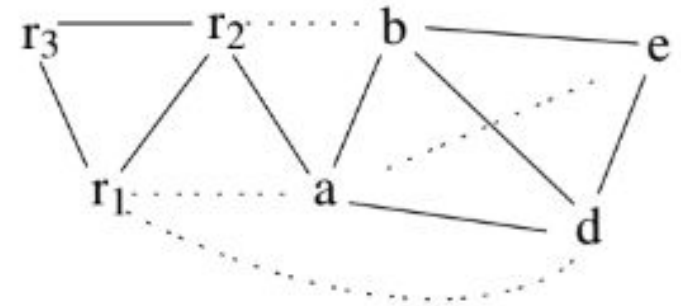
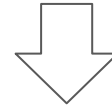
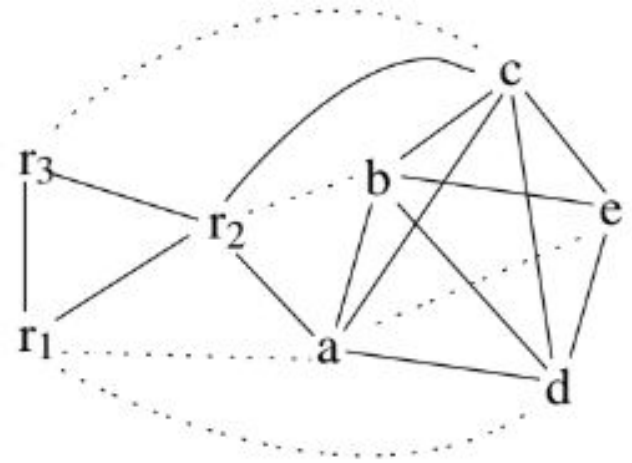


11.3 Precolored Nodes

1.

Spill 후보 : c (거의 사용되지 않기 때문)

Node	Uses+Defs outside loop				Uses+Defs within loop				Degree	Spill priority
<i>a</i>	(	2	+	10 ×	0	)	/	4	=	0.50
<i>b</i>	(	1	+	10 ×	1	)	/	4	=	2.75
<i>c</i>	(	2	+	10 ×	0	)	/	6	=	0.33
<i>d</i>	(	2	+	10 ×	2	)	/	4	=	5.50
<i>e</i>	(	1	+	10 ×	3	)	/	3	=	10.33



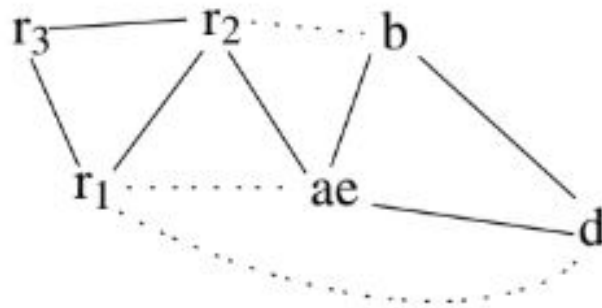
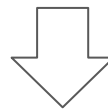
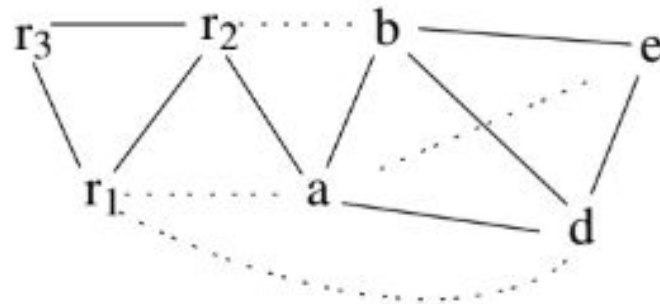
11.3 Precolored Nodes

2. a와 e를 coalesce할 수 있음

: 새 노드 **ae**는 K개보다 적은 수의

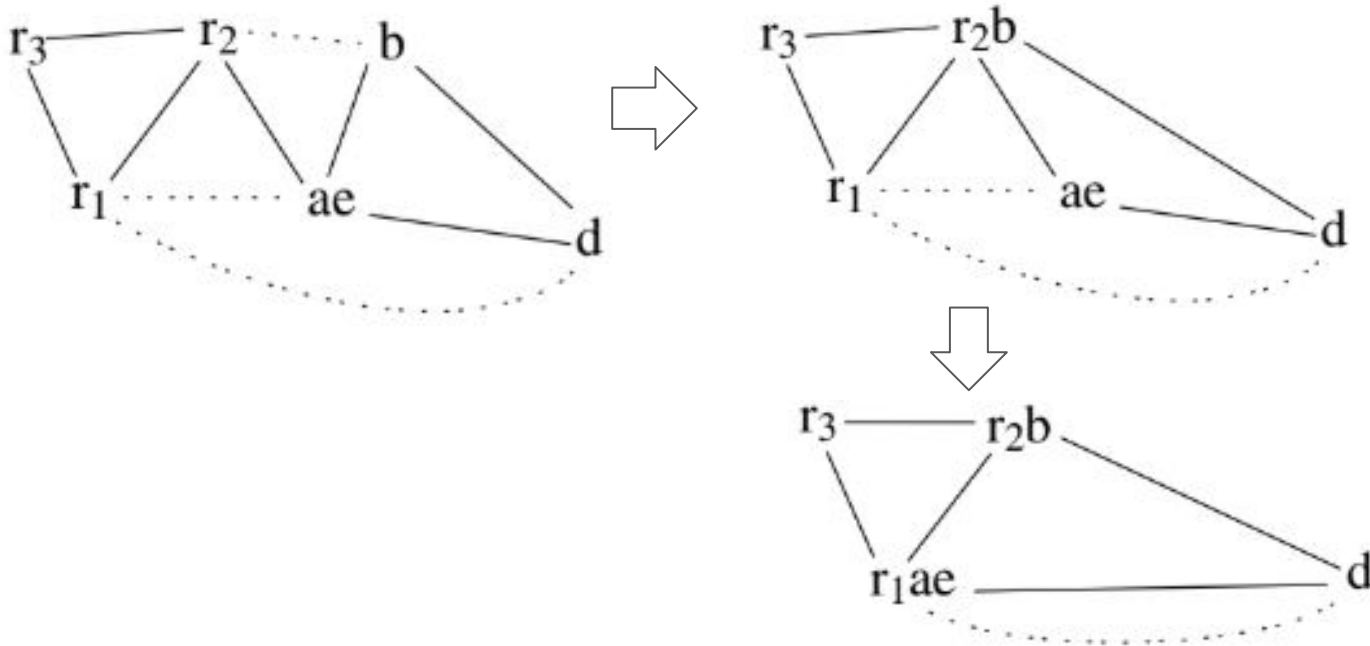
significant-degree node와 인접하므로!

(b, d의 degree = 2 < K)



11.3 Precolored Nodes

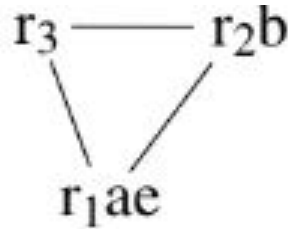
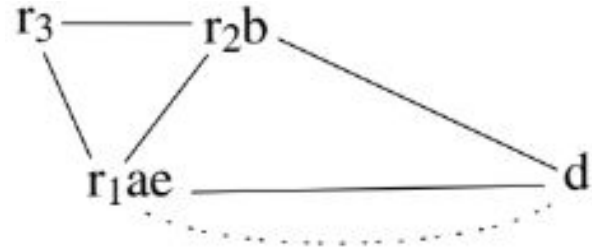
3-4. b와 r2를 합치고, ae와 r1을 합침



11.3 Precolored Nodes

5. r_1ae 와 d 는 간섭하므로 (실선)

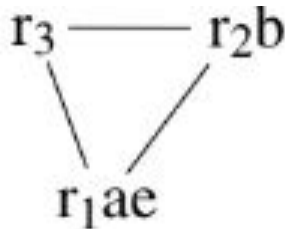
d 를 Simplify함



11.3 Precolored Nodes

6. 이제 그래프에는 **precolored node**만 남아 있음

- **Select** 단계 시작
- **d**를 선택하여 레지스터 **r3**를 할당
- **a**와 **e**는 레지스터 **r1**으로 이미 할당있음
- **b**는 레지스터 **r2**로 이미 할당되어 있음
- **potential spill**로 분류되었던 **c**는 남은 레지스터가 없으므로 **actual spill**됨



11.3 Precolored Nodes

7. 프로그램 재작성 (spill이 발생했으므로)

```
enter:   $c \leftarrow r_3$   
         $a \leftarrow r_1$   
         $b \leftarrow r_2$   
         $d \leftarrow 0$   
         $e \leftarrow a$   
loop:    $d \leftarrow d + b$   
         $e \leftarrow e - 1$   
        if  $e > 0$  goto loop  
         $r_1 \leftarrow d$   
         $r_3 \leftarrow c$   
return  ( $r_1, r_3$  live out)
```

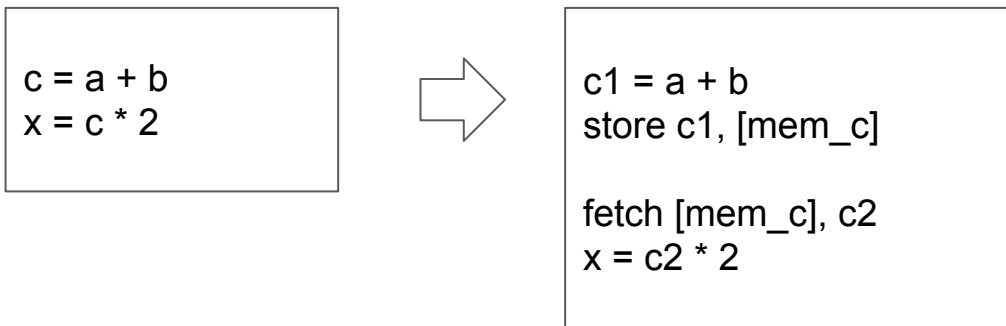


```
enter:   $c_1 \leftarrow r_3$   
         $M[c_{\text{loc}}] \leftarrow c_1$   
         $a \leftarrow r_1$   
         $b \leftarrow r_2$   
         $d \leftarrow 0$   
         $e \leftarrow a$   
loop:    $d \leftarrow d + b$   
         $e \leftarrow e - 1$   
        if  $e > 0$  goto loop  
         $r_1 \leftarrow d$   
         $c_2 \leftarrow M[c_{\text{loc}}]$   
         $r_3 \leftarrow c_2$   
return
```

11.3 Precolored Nodes

7. 프로그램 재작성 (spill이 발생했으므로)

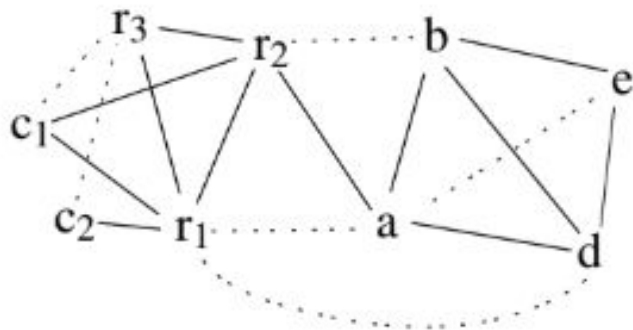
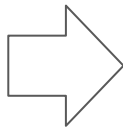
- spill된 임시 변수 **c**의 각 사용 또는 정의 위치마다 새로운 임시 변수 만듦



11.3 Precolored Nodes

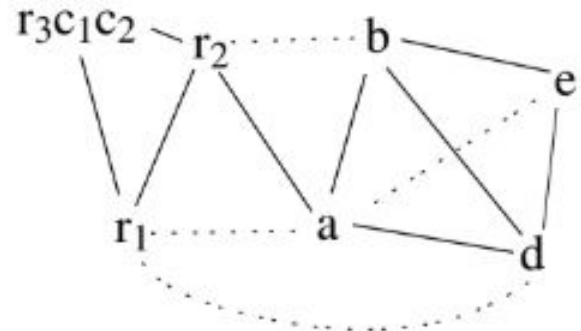
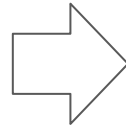
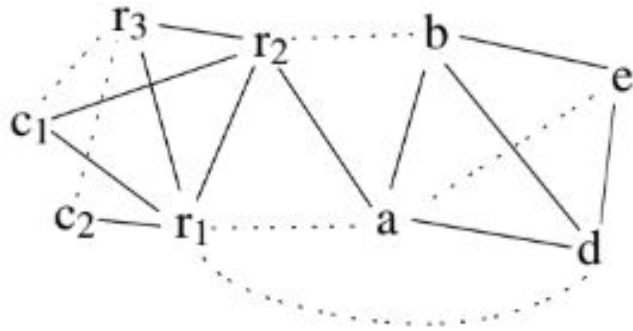
8. 새로운 간섭 그래프 만들기

```
enter:   $c_1 \leftarrow r_3$   
         $M[c_{\text{loc}}] \leftarrow c_1$   
         $a \leftarrow r_1$   
         $b \leftarrow r_2$   
         $d \leftarrow 0$   
         $e \leftarrow a$   
loop:   $d \leftarrow d + b$   
         $e \leftarrow e - 1$   
        if  $e > 0$  goto loop  
         $r_1 \leftarrow d$   
         $c_2 \leftarrow M[c_{\text{loc}}]$   
         $r_3 \leftarrow c_2$   
return
```



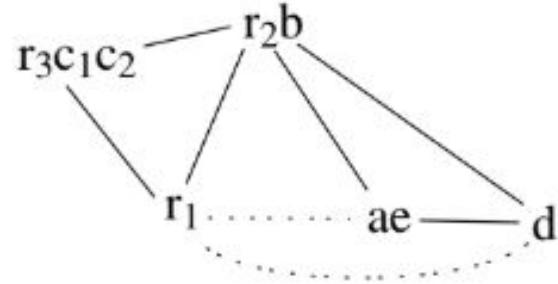
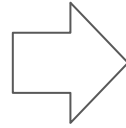
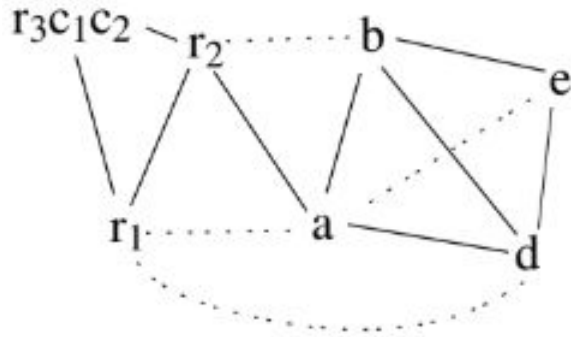
11.3 Precolored Nodes

9. c_1 과 r_3 를 합하고, c_2 와 r_3 를 합침



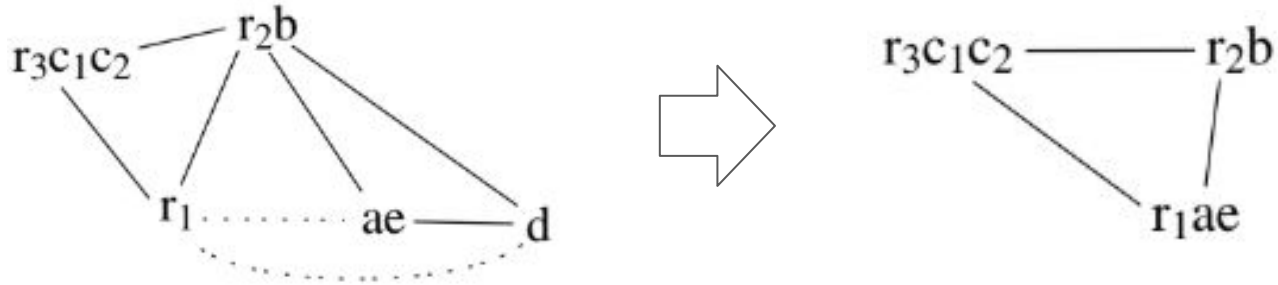
11.3 Precolored Nodes

10. a와 e를 합하고, b와 r2를 합침



11.3 Precolored Nodes

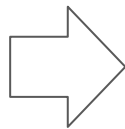
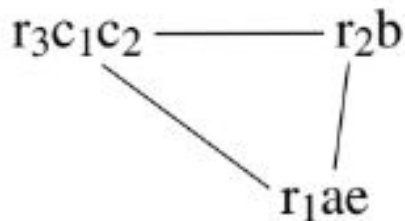
11. ae와 r1을 합하고, d를 Simplify



11.3 Precolored Nodes

12. Select 단계로 스택에서 노드 꺼내며 색칠하기

- d를 레지스터 r3
- 다른 임시 변수는 합쳐졌거나 이미 색칠되었음



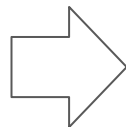
Node	Color
<i>a</i>	<i>r₁</i>
<i>b</i>	<i>r₂</i>
<i>c</i>	<i>r₃</i>
<i>d</i>	<i>r₃</i>
<i>e</i>	<i>r₁</i>

11.3 Precolored Nodes

13. 레지스터 할당에 따라 프로그램 작성

Node	Color
<i>a</i>	<i>r</i> ₁
<i>b</i>	<i>r</i> ₂
<i>c</i>	<i>r</i> ₃
<i>d</i>	<i>r</i> ₃
<i>e</i>	<i>r</i> ₁

```
enter:  c1 ← r3
        M[cloc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if e > 0 goto loop
        r1 ← d
        c2 ← M[cloc]
        r3 ← c2
        return
```

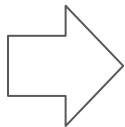


```
enter:  r3 ← r3
        M[cloc] ← r3
        r1 ← r1
        r2 ← r2
        r3 ← 0
        r1 ← r1
loop:   r3 ← r3 + r2
        r1 ← r1 - 1
        if r1 > 0 goto loop
        r1 ← r3
        r3 ← M[cloc]
        r3 ← r3
        return
```

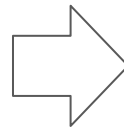
11.3 Precolored Nodes

14. source와 destination이 동일한 move 명령어 제거(coalescing 결과)

```
enter:   $c_1 \leftarrow r_3$   
         $M[c_{loc}] \leftarrow c_1$   
         $a \leftarrow r_1$   
         $b \leftarrow r_2$   
         $d \leftarrow 0$   
         $e \leftarrow a$   
loop:    $d \leftarrow d + b$   
         $e \leftarrow e - 1$   
        if  $e > 0$  goto loop  
         $r_1 \leftarrow d$   
         $c_2 \leftarrow M[c_{loc}]$   
         $r_3 \leftarrow c_2$   
        return
```



```
enter:   $r_3 \leftarrow r_3$   
         $M[c_{loc}] \leftarrow r_3$   
         $r_1 \leftarrow r_1$   
         $r_2 \leftarrow r_2$   
         $r_3 \leftarrow 0$   
         $r_1 \leftarrow r_1$   
loop:    $r_3 \leftarrow r_3 + r_2$   
         $r_1 \leftarrow r_1 - 1$   
        if  $r_1 > 0$  goto loop  
         $r_1 \leftarrow r_3$   
         $r_3 \leftarrow M[c_{loc}]$   
         $r_3 \leftarrow r_3$   
        return
```



```
enter:   $M[c_{loc}] \leftarrow r_3$   
         $r_3 \leftarrow 0$   
loop:    $r_3 \leftarrow r_3 + r_2$   
         $r_1 \leftarrow r_1 - 1$   
        if  $r_1 > 0$  goto loop  
         $r_1 \leftarrow r_3$   
         $r_3 \leftarrow M[c_{loc}]$   
        return
```

11.4 Graph Coloring Implementation

교재 11.4 참고

실습: Graph Coloring

그래프 컬러링 기반 레지스터 할당을 두 개의 모듈로 구현

- **Color**: 그래프 컬러링 자체만 수행하는 모듈
- **RegAlloc**: **spilling**을 관리하고, **Color**를 서브루틴으로 호출하는 모듈

(참고) 간단 구현을 위해 **spilling**이나 **coalescing**은 구현하지 않아도 됨

상세 내용은 교재 **P.244** 실습 설명 참고

실습: Graph Coloring

그래프 컬러링 기반 레지스터 할당을 두 개의 모듈로 구현

```
package RegAlloc;

public class RegAlloc implements Temp.TempMap {
    public Assem.InstrList instrs;
    public String tempMap(Temp temp);
    public RegAlloc(Frame.Frame f, Assem.InstrList il);
}

class Color implements TempMap {
    public TempList spills();
    public String tempMap(Temp t);
    public Color(InterferenceGraph ig,
                 TempMap initial,
                 TempList registers);
}
```


실습: Graph Coloring

Color

- 입력: interference graph, initial allocation, registers
- 출력: TempMap, spills list

RegAlloc

- spilling 관리
- Color를 subroutine으로 호출

실습: Graph Coloring

단순 구현

- coalescing과 freezeWorklist 없음
- simplifyWorklist 하나만 사용

simplifyWorklist

- degree < K인 non-precolored node 저장

spill 후보는 simplifyWorklist 리스트에 원소가 없으면 전체 그래프에서 선택

선형 스캔 알고리즘(Linear scan algorithm)

- 프로그램을 선형화(즉, 프로그램을 그래프가 아니라 명령어들의 선형 순서로 표현)
- 각 변수마다 하나의 고유한 **live range**(해당 변수가 살아 있는 첫 번째 명령어부터 마지막 명령어까지의 구간)를 계산
- **interval**들을 시작 지점이 증가하는 순서로 정렬한 뒤, 차례대로 순회하면서 레지스터를 할당. 어떤 **interval**이 시작되면 사용 가능한 다음 레지스터를 할당하고, 어떤 **interval**이 끝나면 그 레지스터를 해제한다.
- 사용 가능한 레지스터가 없으면, 현재 활성화된 **live range**들 중 가장 늦게 끝나는 것을 선택하여 그 변수는 **spill**시킴.

<https://cs420.epfl.ch/> Advanced Compiler Construction 강의 자료에서 발췌

선형 스캔 알고리즘(Linear scan algorithm)

gcd 함수에서 레지스터를 할당

- 먼저 할당 가능한 레지스터가 4개인 경우, 그 다음 3개인 경우

Program

```
1 gcd:  v0 ← RLK
2      v1 ← R1
3      v2 ← R2
4 loop: v3 ← done
5      if v2=0 goto v3
6      v4 ← v2
7      v2 ← v1 % v2
8      v1 ← v4
9      v5 ← loop
10     goto v5
11 done: R1 ← v1
12     goto v0
```

Live ranges

v₀: [1⁺, 12⁻]

v₁: [2⁺, 11⁻]

v₂: [3⁺, 10⁺]

v₃: [4⁺, 5⁻]

v₄: [6⁺, 8⁻]

v₅: [9⁺, 10⁻]

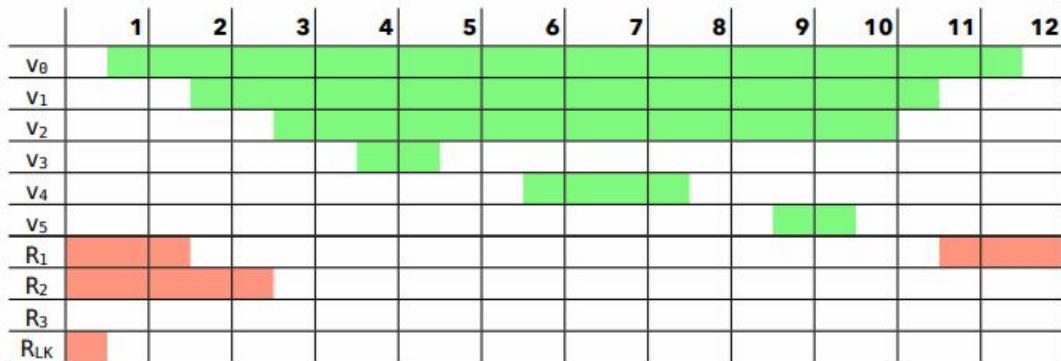
Notation:

i^+ entry of instr. i

i^- exit of instr. i

선형 스캔 알고리즘(Linear scan algorithm)

할당 가능한 레지스터가 4개인 경우 (spill 없음)



time active intervals

allocation

1⁺ [1⁺, 12⁻]

$v_0 \rightarrow R_3$

2⁺ [2⁺, 11⁻], [1⁺, 12⁻]

$v_0 \rightarrow R_3, v_1 \rightarrow R_1$

3⁺ [3⁺, 10⁺], [2⁺, 11⁻], [1⁺, 12⁻]

$v_0 \rightarrow R_3, v_1 \rightarrow R_1, v_2 \rightarrow R_2$

4⁺ [4⁺, 5⁻], [3⁺, 10⁺], [2⁺, 11⁻], [1⁺, 12⁻]

$v_0 \rightarrow R_3, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_3 \rightarrow R_{LK}$

6⁺ [6⁺, 8⁻], [3⁺, 10⁺], [2⁺, 11⁻], [1⁺, 12⁻]

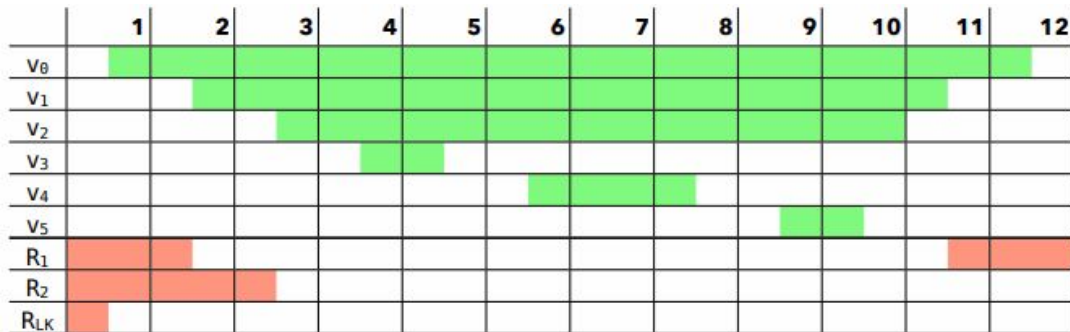
$v_0 \rightarrow R_3, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_4 \rightarrow R_{LK}$

9⁺ [9⁺, 10⁻], [3⁺, 10⁺], [2⁺, 11⁻], [1⁺, 12⁻]

$v_0 \rightarrow R_3, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_5 \rightarrow R_{LK}$

선형 스캔 알고리즘(Linear scan algorithm)

할당 가능한 레지스터가 3개인 경우 (v_0 를 spill)



time active intervals

allocation

$1^+ [1^+, 12^-]$	$v_0 \rightarrow R_{LK}$
$2^+ [2^+, 11^-], [1^+, 12^-]$	$v_0 \rightarrow R_{LK}, v_1 \rightarrow R_1$
$3^+ [3^+, 10^-], [2^+, 11^-], [1^+, 12^-]$	$v_0 \rightarrow R_{LK}, v_1 \rightarrow R_1, v_2 \rightarrow R_2$
$4^+ [4^+, 5^-], [3^+, 10^-], [2^+, 11^-]$	$v_0 \rightarrow S, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_3 \rightarrow R_{LK}$
$6^+ [6^+, 8^-], [3^+, 10^-], [2^+, 11^-]$	$v_0 \rightarrow S, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_4 \rightarrow R_{LK}$
$9^+ [9^+, 10^-], [3^+, 10^-], [2^+, 11^-]$	$v_0 \rightarrow S, v_1 \rightarrow R_1, v_2 \rightarrow R_2, v_5 \rightarrow R_{LK}$

선형 스캔 알고리즘(Linear scan algorithm)

알고리즘 개선 방향

- 가상 레지스터의 **liveness** 정보는 하나의 **interval**이 아니라, 서로 겹치지 않는 여러 **interval**들의 **sequence**로 표현
- 가상 레지스터는 전체 생존 기간 동안이 아니라, 그중 일부 구간에서만 **spill**되도록
- **spill**할 가상 레지스터를 선택할 때 더 정교한 휴리스틱을 사용